

Claude DSA

Level 2: Basic Data Structures

4. Arrays

-- Array basics and operations

An array is a fixed-size, contiguous block of memory. Each element is stored right next to the previous one, and **arr** itself is a pointer to the first element.

1. Declaration & Initialization

```
// Fixed size (stack)
int arr[5]; // uninitialized (garbage values)
int arr[5] = {10, 20, 30, 40, 50}; // fully initialized
int arr[5] = {1, 2}; // rest default to 0 → {1,2,0,0,0}
int arr[] = {1, 2, 3}; // size inferred → 3
int arr[5] = {}; // all zeros
// 2D array
int matrix[3][4]; // 3 rows, 4 cols
int matrix[2][3] = {{1,2,3},{4,5,6}};
```

2. Access & Modification

```
int arr[5] = {10, 20, 30, 40, 50};
// Read
cout << arr[2]; // 30
// Write
arr[2] = 99; // {10, 20, 99, 40, 50}
// Pointer arithmetic (same as arr[i])
cout << *(arr + 2); // 99
```

3. Common Operations

```
#include <iostream>
#include <algorithm>
#include <numeric>
using namespace std;
int arr[] = {5, 3, 8, 1, 9, 2};
int n = sizeof(arr) / sizeof(arr[0]); // ← size trick
// Traverse
for (int i = 0; i < n; i++)
    cout << arr[i] << " ";
// Range-based for (C++11)
for (int x : arr)
    cout << x << " ";
// Sort
sort(arr, arr + n); // ascending: 1 2 3 5 8 9
sort(arr, arr + n, greater<int>()); // descending: 9 8 5 3 2 1
// Min / Max
cout << *min_element(arr, arr + n); // 1
cout << *max_element(arr, arr + n); // 9
// Sum
cout << accumulate(arr, arr + n, 0); // 28
// Binary search (array must be sorted first)
sort(arr, arr + n);
bool found = binary_search(arr, arr + n, 5); // true
// Find index of element
int* pos = find(arr, arr + n, 5);
int idx = pos - arr; // idx = 3 (0-based)
// Reverse
reverse(arr, arr + n);
// Fill with a value
```

```
fill(arr, arr + n, 0);
```

4. Passing Arrays to Functions

// Arrays decay to pointers — always pass size separately

```
void printArray(int arr[], int n) { // same as int* arr
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
}
```

// Modifying inside function affects original (no copy made)

```
void doubleAll(int arr[], int n) {
    for (int i = 0; i < n; i++)
        arr[i] *= 2;
}
```

```
int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int n = sizeof(arr) / sizeof(arr[0]);
    doubleAll(arr, n); // arr is now {2, 4, 6, 8, 10}
}
```

5. 2D Array Operations

```
int mat[3][3] = {{1,2,3},{4,5,6},{7,8,9}};
```

```
int rows = 3, cols = 3;
```

// Traverse

```
for (int i = 0; i < rows; i++)
    for (int j = 0; j < cols; j++)
        cout << mat[i][j] << " ";
```

// Row sum

```
for (int i = 0; i < rows; i++) {
    int sum = 0;
    for (int j = 0; j < cols; j++) sum += mat[i][j];
    cout << "Row " << i << " sum: " << sum << "\n";
}
```

```
/*-----Start-----*/
```

Access $O(1)$

Read element at index i directly.

```
arr = [10, 25, 47, 8, 33]
```

```
val = arr[2] # → 47
```

Update $O(1)$

Overwrite value at a known index.

```
arr[2] = 99
```

```
# arr → [10,25,99,8,33]
```

Search (linear) $O(n)$

Scan every element to find a value.

```
def search(arr, target):
```

```
    for i, v in enumerate(arr):
```

```
        if v == target:
```

```
            return i
```

```
    return -1
```

Insert at end $O(1)^*$

Append — no shifting needed. * Amortized.

```
arr.append(55)
```

```
# arr → [10,25,47,8,33,55]
```

Insert at middle / beginning $O(n)$

All elements after the insert position must shift right — expensive.

```
arr.insert(2, 99)  # insert 99 at index 2
# [10,25, → 99 ← , 47,8,33]  everything from index 2 shifted right
```

Delete from middle $O(n)$

Elements after deleted index must shift left to fill the gap.

```
arr.pop(2)        # remove index 2
arr.remove(47)     # remove first occurrence of value 47
```

Length $O(1)$

Array stores its size — instant lookup.

```
n = len(arr)  # → 5
```

Slicing $O(k)$

k = number of elements in slice.

```
sub = arr[1:4]
# → [25, 47, 8]
```

Quick-reference complexity

Operation	Best	Worst
Access / Update	$O(1)$	$O(1)$
Insert/Delete (end)	$O(1)^*$	$O(1)^*$
Insert/Delete (mid)	$O(n)$	$O(n)$
Search (unsorted)	$O(1)$	$O(n)$
Search (sorted)	$O(1)$	$O(\log n)$

Given an array [3, 1, 4, 1, 5, 9, 2, 6], write a function that returns the index of the maximum element. What is the time complexity?

```
#include <iostream>
#include <algorithm>
#include <cmath>
using namespace std;
int main()
{
    int arr[] = {3, 1, 4, 1, 5, 9, 2, 6};
    int s = sizeof(arr)/sizeof(arr[0]);
    int maxi = arr[0];
    int maxIdx = 0;
    for(int i=1;i<s;i++)
    {
        if(arr[maxIdx] < arr[i])
            maxIdx = i;
    }
    cout << "Index: " << maxIdx << endl;
    return 0;
}
```

Output: ./array_1P

Index: 5

-- Static vs dynamic arrays

what happens when you run out of space?

A static array says "tough luck" — its size is fixed at compile time, carved into the stack. A dynamic array says "hold on, let me get a bigger box" — it lives on the heap and can grow. Let's see exactly how each works.

Static arrays are what you've been using — `int arr[5]`. Size is decided before the program runs, stored on the stack, zero overhead. The catch: you must know the size upfront, and you can never change it.

The key insight on the right side: vector stores just three things on the stack — a pointer, a size, and a capacity. The actual data lives on the heap. Those dashed boxes are pre-allocated spare slots, which brings us to the most important mechanism:

doubling.

When a vector runs out of capacity, it doesn't just add one slot. It allocates a brand new block of memory that's double the current size, copies everything over, then frees the old block. This is why `push_back` is $O(1)$ *amortized* — most of the time it's instant, but occasionally it pays a $O(n)$ copy cost.

	Static int arr[N]	Dynamic vector<int>
Memory location	Stack	Heap
Size at compile time	Known, fixed	Not needed
Can grow/shrink	No	Yes
Access speed	Slightly faster	Same $O(1)$
Memory overhead	Zero	~24 bytes (ptr+size+cap)
Risk of overflow	Yes (silent!)	No (auto-grows)
Use when	Size known, perf critical	General purpose

Given: `vector<int> v = {5, 3, 8, 1, 9, 2}`; Without using any built-in sort or max function: 1. Find and print the maximum element 2. Move it to the front of the vector Print the final vector. Expected output: 9 5 3 8 1 2

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v = {5, 3, 8, 1, 9, 2};
    int maxVal = 0, maxIdx = 0;
    //1. Find and print the maximum element
    for(int i=0;i<v.size();i++)
    {
        if(maxVal < v[i])
        {
            maxVal = v[i];
            maxIdx = i;
        }
    }
    cout << "MaxVal:- " << maxVal << endl;
    //2. Move it to the front of the vector
    /*int temp = v[0];
    v[0] = v[maxIdx];
    v[maxIdx] = temp;*/
    swap(v[0], v[maxIdx]); // prefer this
    //3. Print the final vector.
    cout << "Expected output: ";
    for(int i=0;i<v.size();i++)
    {
        cout << v[i] << " ";
    }
}
```

```

        cout << endl;
        return 0;
    }
    Output: ./array_2P
    MaxVal:- 9
    Expected output: 9 3 8 1 5 2

```

-- 1D Array - Traversal

What is traversal?

- Visiting each element of an array exactly once (or in a pattern) to read or process it
- All traversals are $O(n)$ time — you touch n elements
- $O(1)$ space — no extra memory needed unless you're building output
- The loop variable i is your pointer into memory: $\text{arr}[i] = \text{base} + i \times \text{size}$

Forward most common

```
for(int i=0; i<n; i++)
```

- Left $\rightarrow$ right, every index
- Use for: sum, max, search, print

Backward prefix/suffix

```
for(int i=n-1; i>=0; i--)
```

- Right $\rightarrow$ left
- Use for: suffix arrays, reverse print

Two pointer interview fav

```
int l=0, r=n-1;
```

```
while(l < r){ l++; r--; }
```

- Both ends move inward
- Use for: palindrome, pair sum, reversal

Step skip less common

```
for(int i=0; i<n; i+=k)
```

- Jump k indices at a time
- Use for: even/odd index, chunks

KEY INTERVIEW POINTS

Time complexity	$O(n)$ — always for full traversal
Space complexity	$O(1)$ — no extra array needed
Initialise from	$\text{arr}[0]$, never a hardcoded 0 or -1
Loop bound	$i < n$ (not $\leq n$ — off-by-one!)
$\text{v.size}()$ type	unsigned — cast to (int) when comparing
Two pointer condition	$l < r$ (not $l \leq r$ unless you want mid)
Swap without temp	$a \wedge= b; b \wedge= a; a \wedge= b;$ (XOR trick)

COMMON MISTAKES

- $i \leq n$ instead of $i < n \rightarrow$ accesses $\text{arr}[n]$, undefined behaviour
- Initialising max/min to 0 $\rightarrow$ fails for all-negative arrays
- Forgetting to increment both pointers in two-pointer $\rightarrow$ infinite loop
- Modifying the array inside the loop while also reading it $\rightarrow$ logic bugs

RECOGNISE THE PATTERN IN PROBLEMS

"Find max / min / sum"	$\rightarrow$ Forward traversal
"Reverse the array"	$\rightarrow$ Two pointer or backward
"Check palindrome"	$\rightarrow$ Two pointer ($l < r$)
"Pair with target sum"	$\rightarrow$ Two pointer on sorted array

"Every other element" → Step skip (i += 2)
"Suffix/prefix array" → Backward then forward

Given: vector<int> v = {1, 2, 3, 4, 5, 6, 7, 8}; Using two pointer traversal — reverse the array in-place. Do NOT use any built-in reverse function. Expected output: 8 7 6 5 4 3 2 1

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v = {1, 2, 3, 4, 5, 6, 7, 8};
    int l = 0, r = v.size()-1;
    while(l < r)
    {
        swap(v[l], v[r]);
        l++;
        r--;
    }
    cout << "Expected output: ";
    for(int i=0;i<v.size();i++)
        cout << v[i] << " ";
    cout << endl;
    return 0;
}
```

Output: ./array_3P

Expected output: 8 7 6 5 4 3 2 1

Find second largest element (1 Pass)*

Given an array, find the second largest element without sorting.

```
#include <iostream>
#include <vector>
#include <climits>
using namespace std;
int main()
{
    vector<int> v{12, 35, 1, 10, 34, 1};
    int lar = INT_MIN;
    int second_lar = INT_MIN;
    //2 pass
    /* for(int i=1;i<v.size();i++)
    {
        if(lar < v[i])
            lar = v[i];
    }
    cout << "Largest Value:- " << lar << endl;
    for(int i=0;i<v.size();i++)
    {
        if(second_lar < v[i] && v[i] != lar )
            second_lar = v[i];
    }
    cout << "2nd Largest Value:- " << second_lar << endl; */
    for(int i=0;i<v.size();i++)
    {
        if(v[i] > lar)
        {
            second_lar = lar;
            lar = v[i];
        }
    }
    cout << "2nd Largest Value:- " << second_lar << endl;
}
```

```

        lar = v[i];
    }
    else if(v[i] > second_lar && v[i] != lar)
        second_lar = v[i];
    }
    cout << "Largest Value:- " << lar << endl;
    cout << "2nd Largest Value:- " << second_lar << endl;
    return 0;
}
Output: ./array_4P
Largest Value:- 35
2nd Largest Value:- 34

```

Count occurrences of a value

Count how many times a given target appears in the array.

```

#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v{1, 4, 2, 4, 3, 4};
    int target = 4;
    int cnt = 0;
    for(int i=0;i<v.size();i++)
    {
        if(v[i] == target)
            cnt += 1;
    }
    cout << "Count:- " << cnt << endl;
    return 0;
}
Output: ./array_5P
Count:- 3

```

Check if array is sorted

Return true if array is sorted in non-decreasing order, else false.

```

#include <iostream>
#include <vector>
using namespace std;
bool isSorted(vector<int>& vec)
{
    for(int i=1;i<vec.size();i++)
    {
        if(vec[i-1] > vec[i])
            return false;
    }
    return true;
}
int main()
{
    //vector<int> v{1, 2, 3, 4, 5};
    vector<int> v{1, 3, 2, 4, 5};
    bool res = isSorted(v);
    if(res)
        cout << "Vector is Sorted." << endl;
    else
        cout << "Vector is not Sorted." << endl;
    return 0;
}

```

```
}
```

Output: ./array_6P

Vector is not Sorted.

Check palindrome array*

Return true if array reads the same forwards and backwards.

```
#include <iostream>
#include <vector>
using namespace std;
bool isPalindrome(vector<int>& v)
{
    int l = 0, r = v.size()-1;
    while(l<r)
    {
        if(v[l] != v[r])
            return false;
        l++;
        r--;
    }
    return true;
}
int main()
{
    //vector<int> v{1, 2, 3, 2, 1};
    vector<int> v{1, 2, 3, 4, 5};
    bool res = isPalindrome(v);
    if(res)
        cout << "Vector is Palindrome.\n";
    else
        cout << "Vector is not Palindrome.\n";

    return 0;
}
```

Output: ./array_7P

Vector is not Palindrome.

Move all zeros to end*

Move all 0s to the end while maintaining relative order of non-zero elements. In-place.

```
#include <iostream>
#include <vector>
using namespace std;
void move_zero_to_end(vector<int>& v)
{
    int n = v.size();
    int j = 0;
    for(int i=0;i<n;i++)
    {
        if(v[i] != 0)
        {
            swap(v[i], v[j]);
            j++;
        }
    }
}
int main()
{
    vector<int> v{0, 1, 0, 3, 12};
```



```

        move_zero_to_end(v);
        cout << "Output: ";
        for(int i=0;i<(int)v.size();i++)
            cout << v[i] << " ";
        cout << endl;
        return 0;
    }
    Output: ./array_8P
    Output: 1 3 12 0 0

```

Left rotate array by k positions*

Rotate array to the left by k steps. In-place preferred.

```

#include <iostream>
#include <vector>
using namespace std;
void interchange(vector<int>& v, int s, int e)
{
    int l = s, r = e;
    while(l < r)
    {
        swap(v[l], v[r]);
        l++;
        r--;
    }
}
int main()
{
    vector<int> v{1, 2, 3, 4, 5};
    int k = 2;
    int n = (int)v.size();
    interchange(v, 0, k-1);
    interchange(v, k, n-1);
    interchange(v, 0, n-1);
    cout << "Result: ";
    for(int i=0;i<n;i++)
        cout << v[i] << " ";
    cout << endl;
    return 0;
}
    Output: ./array_9P
    Result: 3 4 5 1 2

```

Pair with given sum (sorted array)*

Find if any two elements sum to a target. Array is sorted.

```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
vector<pair<int, int>> pair_out(vector<int>& v, int tar)
{
    vector<pair<int, int>> res;
    int l = 0, r = v.size()-1;
    while(l < r)
    {
        int sum = v[l] + v[r];
        if(sum == tar)
        {
            res.push_back({v[l], v[r]});

```

```

        l++;
        r--;
    }
    else if(sum < tar)
        l++;
    else
        r--;
    }
    return res;
}
int main()
{
    vector<int> v{1, 2, 3, 4, 5, 6};
    int tar = 6;
    vector<pair<int, int>> res = pair_out(v, tar);
    for(auto p : res)
        cout << "{" << p.first << ", " << p.second << "}" ";
    cout << endl;
    return 0;
}
Output: ./array_10P
{1, 5} {2, 4}

```

Find the missing number (1 to n)

Array has n-1 elements from 1 to n with one missing. Find it.

```

#include <iostream>
#include <vector>
using namespace std;
int missing_num(vector<int>& v, int n)
{
    int re = 0, sum = 0;
    for(int i=0;i<(int)v.size();i++)
    {
        sum += v[i];
    }
    int actualSum = n*(n+1)/2;
    re = actualSum - sum;
    return re;
}
int main()
{
    vector<int> v{1, 2, 4, 5, 6};
    int n = 6;
    int res = missing_num(v, n);
    cout << "Result: " << res << endl;
    return 0;
}
Output: ./array_11P
Result: 3

```

-- 1D Array - Insertion & Deletion in Arrays

The core idea is simple: arrays live in contiguous memory — so inserting or deleting in the middle forces you to **shift elements**. That shifting cost is what makes these operations $O(n)$.

THEORY

Why insertion and deletion are expensive in arrays

- Array elements sit in *contiguous memory* — no gaps allowed
- Inserting in the middle creates a gap → must shift everything right to fill it
- Deleting from the middle creates a hole → must shift everything left to close it
- Only end operations are $O(1)$ — no shifting needed

ALL CASES AT A GLANCE

Insertion shifts right

At end $O(1)$

At middle (pos i) $O(n-i)$

At beginning $O(n)$ worst

// insert at pos — shift right from end

```
for(int i=n-1; i>pos; i--)
```

```
    v[i] = v[i-1];
```

```
v[pos] = val;
```

Deletion shifts left

From end $O(1)$

From middle (pos i) $O(n-i)$

From beginning $O(n)$ worst

// delete at pos — shift left from pos

```
for(int i=pos; i<n-1; i++)
```

```
    v[i] = v[i+1];
```

```
v.pop_back();
```

KEY INTERVIEW POINTS

Insertion direction Loop runs **right to left** (end → pos) to avoid overwriting

Deletion direction Loop runs **left to right** (pos → end) to fill the gap

After deletion Always `pop_back()` — last slot is a duplicate

Before insertion Check capacity / `push_back(0)` to make space first

vector built-ins `v.insert(v.begin()+pos, val)` and `v.erase(v.begin()+pos)`

Both are $O(n)$ Because of shifting — this is why linked lists exist

COMMON MISTAKES

- Insertion loop going left to right — overwrites data before shifting it
- Forgetting `pop_back()` after deletion — leaves a duplicate at the end
- Not checking bounds before inserting — pos must be in $[0, n]$
- Not growing vector before manual insertion — writing to `arr[n]` is UB in raw arrays

PATTERN RECOGNITION

"Remove duplicates in-place" → Delete pattern with two pointers

"Insert into sorted array" → Find position + insert with right shift

"Remove element by value" → Find index first, then delete

"Frequent inserts/deletes?" → Arrays are wrong — use linked list

COMPLEXITY SUMMARY

Operation	Time	Space
Insert at end	$O(1)^*$	$O(1)$
Insert at pos i	$O(n-i)$	$O(1)$
Delete from end	$O(1)$	$O(1)$
Delete from pos i	$O(n-i)$	$O(1)$

* amortized for vector — occasional $O(n)$ resize

1) Insert into sorted array*

Insert a value into a sorted array maintaining sorted order. No built-in insert().

Pattern: find position where val fits, shift right from end → place val

Input: {1, 3, 5, 6}, val=2

```
#include <iostream>
#include <vector>
using namespace std;
void insert_ele(vector<int>& v, int va)
{
    int s = v.size();
    v.push_back(0);
    int pos = 0;
    while(pos < s && v[pos] < va)
        pos++;
    for(int i=s;i>pos;i--)
        v[i] = v[i-1];
    v[pos] = va;
}
int main()
{
    vector<int> v{1, 3, 5, 6};
    int val = 2;
    insert_ele(v, val);
    cout << "Output:- ";
    for(int i=0;i<v.size();i++)
        cout << v[i] << " ";
    cout << endl;
    return 0;
}
Output: ./array_12P
Output:- 1 2 3 5 6
```

2) Delete first occurrence of value

Remove the first occurrence of a given value from the array in-place.

Pattern: find index of first occurrence, then shift left from that position

Input: {4, 2, 7, 2, 9}, val=2

```
#include <iostream>
#include <vector>
using namespace std;
void Delete_first_occurrence(vector<int>& v, int val)
{
    int n = v.size();
    int l = -1;
    for(int i=0;i<n;i++)
    {
        if(v[i] == val)
        {
            l = i;
            break;
        }
    }
}
```

```

    }
    if(l == -1) return;
    for(int i=l;i<n-1;i++)
        v[i] = v[i+1];
    v.pop_back();
}
int main()
{
    vector<int> v{4, 2, 7, 2, 9};
    int val = 2;
    Delete_first_occurrence(v, val);
    cout << "Output:- ";
    for(int i=0;i<v.size();i++)
        cout << v[i] << " ";
    cout << endl;
    return 0;
}
Output: ./array_13P
Output:- 4 7 2 9

```

3) Delete all occurrences of value*

Remove every occurrence of a given value from the array in-place.

Pattern: two pointers — j tracks next write position, skip val, copy rest

Input: {3, 2, 3, 5, 3}, val=3

```

#include <iostream>
#include <vector>
using namespace std;
void Delete_all_occurrences(vector<int>& v, int val)
{
    int n = v.size();
    int j = 0;
    for(int i=0;i<n;i++)
    {
        if(v[i] != val)
        {
            v[j] = v[i];
            j++;
        }
    }
    v.resize(j);
}
int main()
{
    vector<int> v{3, 2, 3, 5, 3};
    int val = 3;
    Delete_all_occurrences(v, val);
    cout << "Output: ";
    for(int i=0;i<v.size();i++)
        cout << v[i] << " ";
    cout << endl;
    return 0;
}
Output: ./array_14P
Output: 2 5

```

4) Remove duplicates from sorted array*

Remove duplicates in-place from a sorted array. Return new length. Order must be preserved.

Pattern: two pointers — j=0 is write pointer, i scans forward, copy only when v[i] != v[j]

Input: {1, 1, 2, 3, 3, 4}

```
#include <iostream>
#include <vector>
using namespace std;
void Remove_duplicates(vector<int>& v)
{
    int n = v.size();
    int j = 0;
    for(int i=1;i<n;i++)
    {
        if(v[i] != v[j])
        {
            j++;
            v[j] = v[i];
        }
    }
    v.resize(j+1);
}
int main()
{
    vector<int> v{1, 1, 2, 3, 3, 4};
    Remove_duplicates(v);
    cout << "Output: ";
    for(int i=0;i<v.size();i++)
        cout << v[i] << " ";
    cout << endl;
    return 0;
}
Output: ./array_15P
Output: 1 2 3 4
```

5) Insert at every kth position

Insert value 0 after every k elements in the array.

Pattern: build result vector — copy k elements, insert 0, repeat

Input: {1,2,3,4,5,6}, k=2

```
#include <iostream>
#include <vector>
using namespace std;
vector<int> Insert_at_every_kth_pos(vector<int>& v, int k)
{
    vector<int> res;
    int n = v.size();
    for(int i=0;i<n;i++)
    {
        res.push_back(v[i]);
        if((i+1)%k == 0)
        {
            res.push_back(0);
        }
    }
    return res;
}
int main()
{
    vector<int> v{1, 2, 3, 4, 5, 6};
    int k = 2;
```

```

vector<int> res = Insert_at_every_kth_pos(v, k);
for(int i=0;i<res.size();i++)
    cout << res[i] << " ";
cout << endl;
return 0;
}

```

Output: ./array_16P

1 2 0 3 4 0 5 6 0

6) Merge two sorted arrays in-place*

Given two sorted arrays, merge arr2 into arr1 in-place. arr1 has enough space.

Pattern: fill from the END of arr1 backwards — compare largest from each, place last

Input: arr1={1,3,5,0,0,0} m=3 arr2={2,4,6} n=3

```

#include <iostream>
#include <vector>
using namespace std;
void Merge_two_sorted(vector<int>& v1, int x, vector<int>& v2, int y)
{
    int i = x - 1;
    int j = y - 1;
    int k = x + y - 1;
    while(i >= 0 && j >= 0)
    {
        if(v1[i] > v2[j])
        {
            v1[k] = v1[i];
            i--;
        }
        else
        {
            v1[k] = v2[j];
            j--;
        }
        k--;
    }
    while(j >= 0)
    {
        v1[k] = v2[j];
        j--;
        k--;
    }
}
int main()
{
    vector<int> v1{1, 3, 5, 0, 0, 0};
    int m = 3;
    vector<int> v2{2, 4, 6};
    int n = 3;
    Merge_two_sorted(v1, m, v2, n);
    for(int i=0;i<v1.size();i++)
        cout << v1[i] << " ";
    cout << endl;
    return 0;
}

```

Output: ./array_17P

1 2 3 4 5 6

7) Remove duplicates from unsorted array

Remove duplicates from an unsorted array. Keep first occurrence. Maintain order.

Pattern: use a boolean seen[] array or unordered_set to track visited values

Input: {3, 1, 4, 1, 5, 3, 2}

```
#include <iostream>
#include <vector>
#include <unordered_set>
using namespace std;
void Remove_duplicates(vector<int>& v)
{
    int j = 0;
    unordered_set<int> seen;
    for(int i=0;i<v.size();i++)
    {
        if(seen.find(v[i]) == seen.end())
        {
            seen.insert(v[i]);
            v[j] = v[i];
            j++;
        }
    }
    v.resize(j);
}
int main()
{
    vector<int> v{3, 1, 4, 1, 5, 3, 2};
    Remove_duplicates(v);
    cout << "Output: ";
    for(int i=0;i<v.size();i++)
        cout << v[i] << " ";
    cout << endl;
    return 0;
}
```

Output: ./array_18P

Output: 3 1 4 5 2

8) Rearrange array — negatives first*

Move all negative numbers to the front, positives to the back. Maintain relative order.

Pattern: insertion sort style — when negative found, shift all positives right, insert negative

Input: {1,-1,3,-2,4,-3}

```
#include <iostream>
#include <vector>
using namespace std;
/*void right_shift(vector<int>& v, int s, int e)
{
    for(int i=e;i>=s;i--)
    {
        v[i+1] = v[i];
    }
}
void Rearrange_array(vector<int>& v)
{
    int j = 0;
    int x = v.size();
    v.resize(x+x);
    for(int i=0;i<v.size();i++)
    {
        if(v[i] < 0)
```



```

        {
            int temp = v[i];
            right_shift(v, j, i-1);
            v[j] = temp;
            j++;
        }
    }
    v.resize(x);
}*/
void Rearrange_array(vector<int>& v)
{
    int n = v.size();
    int j = 0; // position to place next negative
    for(int i = 0; i < n; i++)
    {
        if(v[i] < 0)
        {
            int temp = v[i];
            for(int k = i; k > j; k--) // shift positives right
                v[k] = v[k-1];
            v[j] = temp;          // place negative at j
            j++;
        }
    }
}
int main()
{
    vector<int> v{1, -1, 3, -2, 4, -3};
    Rearrange_array(v);
    cout << "Output: ";
    for(int i=0; i<v.size(); i++)
        cout << v[i] << " ";
    cout << endl;
    return 0;
}
Output: ./array_19P
Output: -1 -2 -3 1 3 4

```

9) Sliding window — max of every window of size k*

For each window of size k, find the maximum element.

Pattern: brute force $O(nk)$ first — inner loop finds max in each window of size k

Input: {1,3,-1,-3,5,3,6}, k=3

```

#include <iostream>
#include <vector>
using namespace std;
vector<int> Sliding_window(vector<int>& v, int k)
{
    vector<int> res;
    int n = v.size();
    for(int i=0; i<=n-k; i++)
    {
        int max_val = v[i];
        for(int j=i; j<i+k; j++)
        {
            if(v[j] > max_val)
                max_val = v[j];
        }
        res.push_back(max_val);
    }
}

```

```

    }
    return res;
}
int main()
{
    vector<int> v{1,3,-1,-3,5,3,6};
    int k = 3;
    vector<int> res = Sliding_window(v, k);
    cout << "Output: ";
    for(int i=0;i<res.size();i++)
        cout << res[i] << " ";
    cout << endl;
    return 0;
}
Output: ./array_20P
Output: 3 3 5 5 6

```

10) Trapping rain water*

Given heights of bars, compute how much rain water can be trapped between them.
 Pattern: for each index, water = min(maxLeft, maxRight) - height[i]. Precompute both arrays.

Input: {0,1,0,2,1,0,1,3,2,1,2,1}

```

#include <iostream>
#include <vector>
using namespace std;
int trap_water(vector<int>& v)
{
    int n = v.size();
    if(n == 0) return 0;
    vector<int> maxLeft(n), maxRight(n);
    int res = 0;
    //Max Left
    maxLeft[0] = v[0];
    for(int i=1;i<n;i++)
        maxLeft[i] = max(maxLeft[i-1], v[i]);
    //Max Right
    maxRight[n-1] = v[n-1];
    for(int i=n-2;i>=0;i--)
        maxRight[i] = max(maxRight[i+1], v[i]);
    for(int i=0;i<n;i++)
        res += min(maxLeft[i], maxRight[i]) - v[i];
    return res;
}
int trap_rainwater_optimized(vector<int>& height)
{
    int n = height.size();
    int left = 0, right = n - 1;
    int leftMax = 0, rightMax = 0;
    int water = 0;
    while(left <= right)
    {
        if(height[left] <= height[right])
        {
            if(height[left] >= leftMax)
                leftMax = height[left];
            else
                water += leftMax - height[left];
            left++;
        }
        else
        {
            if(height[right] >= rightMax)
                rightMax = height[right];
            else
                water += rightMax - height[right];
            right--;
        }
    }
    return water;
}

```

```

    }
    else
    {
        if(height[right] >= rightMax)
            rightMax = height[right];
        else
            water += rightMax - height[right];
        right--;
    }
}
return water;
}
int main()
{
    vector<int> v{0,1,0,2,1,0,1,3,2,1,2,1};
    cout << "Output: " << trap_water(v) << endl;
    return 0;
}

```

Output: ./array_21P

Output: 6

- 1D Array - Searching (linear, binary)

Searching means finding whether a value exists in an array — and if so, where. There are two fundamental algorithms: **linear search** and **binary search**. The difference between them is one of the most important ideas in all of DSA.

THEORY

- Linear search — scan every element one by one. Works on any array. $O(n)$.
- Binary search — repeatedly halve the search space. Requires sorted array. $O(\log n)$.
- $O(\log n)$ is extraordinarily fast — on 1 billion elements, binary search takes just 30 steps.
- The mid formula $l + (r-l)/2$ prevents integer overflow vs $(l+r)/2$.

KEY INTERVIEW POINTS

Binary search condition	$\text{while}(l \leq r)$ not $l < r$ — misses single element
Mid formula	$l + (r-l)/2$ — prevents overflow for large $l+r$
Left half	$r = \text{mid}-1$ — exclude mid, already checked
Right half	$l = \text{mid}+1$ — exclude mid, already checked
Not found return	Always return -1 — never 0 or false
Prerequisite	Array MUST be sorted — always state this
Duplicates	Binary search finds ONE occurrence — use lower/upper bound for all

COMPLEXITY COMPARISON

algorithm	time	space	requires
Linear search	$O(n)$	$O(1)$	nothing
Binary search	$O(\log n)$	$O(1)$	sorted array
Binary (recursive)	$O(\log n)$	$O(\log n)$	sorted + call stack

COMMON MISTAKES

- $\text{while}(l < r)$ instead of $l \leq r$ — misses the last element when $l==r$
- $\text{mid} = (l+r)/2$ — overflows when $l+r$ exceeds `INT_MAX`
- $l = \text{mid}$ instead of $l = \text{mid}+1$ — infinite loop when $l==r$
- Using binary search on unsorted array — gives completely wrong results
- Returning 0 when not found — ambiguous (0 is a valid index)

RECOGNISE THE PATTERN

"Search in unsorted array"	→ Linear search $O(n)$
"Search in sorted array"	→ Binary search $O(\log n)$
"Find first/last occurrence"	→ Binary search + lower/upper bound
"Search in rotated sorted array"	→ Modified binary search
"Find peak element"	→ Binary search on answer
"Find sqrt of n"	→ Binary search on answer space
"Minimise maximum / maximise minimum"	→ Binary search on answer

Binary search isn't just for searching arrays — it's a **general problem-solving technique**. Any time your answer lies in a sorted range and you can write a function that says "is this value too big or too small?", you can binary search for it. This is called **binary search on the answer space** and it solves problems like "find square root", "allocate minimum pages", "aggressive cows" — all coming up in your roadmap.

**1. [Easy] Given sorted array {1,3,5,7,9,11,13},
search for target=7. Return its index.**

```
#include <iostream>
#include <vector>
using namespace std;
int binary_search(vector<int>& v, int val)
{
    int l = 0;
    int r = v.size()-1;
    while(l <= r)
    {
        int mid = l + (r - l) / 2;
        if(v[mid] == val)
            return mid;
        else if(v[mid] < val)
            l = mid + 1;
        else
            r = mid - 1;
    }
    return -1;
}
int main()
{
    vector<int> v{1,3,5,7,9,11,13};
    int tar = 7;
    cout << "Index: " << binary_search(v, tar) << endl;
    return 0;
}
```

Output: ./search_1

Index: 3

**2. [Medium] Given sorted array {1,2,2,2,3,4},
find the FIRST occurrence of 2.**

Expected: index 1

```
#include <iostream>
#include <vector>
using namespace std;
int binary_search(vector<int>& v, int val)
{
    int l = 0;
    int pos = -1;
    int r = v.size() - 1;
    while(l <= r)
    {
        int mid = l + (r - l) / 2;
        if(v[mid] == val)
        {
            pos = mid;
            r = mid - 1;
        }
        else if(v[mid] < val)
            l = mid + 1;
        else
            r = mid - 1;
    }
    return pos;
}
int main()
```

```

{
    vector<int> v{1,2,2,2,3,4};
    int tar = 2;
    cout << "Result: " << binary_search(v, tar) << endl;
    return 0;
}
Output: ./search_2
Result: 1

```

3. [Medium] Given sorted array {1,2,2,2,3,4}, find the LAST occurrence of 2.

Expected: index 3

```

#include <iostream>
#include <vector>
using namespace std;
int binary_search(vector<int>& v, int val)
{
    int pos = -1;
    int l = 0;
    int r = v.size()-1;
    while(l <= r)
    {
        int mid = l + (r - l) / 2;
        if(v[mid] == val)
        {
            pos = mid;
            l = mid + 1;
        }
        else if(v[mid] < val)
            l = mid + 1;
        else
            r = mid - 1;
    }
    return pos;
}
int main()
{
    vector<int> v{1,2,2,2,3,4};
    int tar = 2;
    cout << "Result: " << binary_search(v, tar) << endl;
    return 0;
}
Output: ./search_3
Result: 3

```

4. [Hard] Given rotated sorted array {4,5,6,7,0,1,2}, search for target=0.

Expected: index 4

```

#include <iostream>
#include <vector>
using namespace std;
int rotated_sorted(vector<int>& v, int val)
{
    int l = 0;
    int r = v.size() - 1;
    while(l <= r)
    {
        int mid = l + (r - l) / 2;

```

```

        if(v[mid] == val)
            return mid;
        if(v[l] <= v[mid])
        {
            if(v[l] <= val && val < v[mid])
                r = mid - 1;
            else
                l = mid + 1;
        }
        else
        {
            if(v[r] >= val && val > v[mid])
                l = mid + 1;
            else
                r = mid - 1;
        }
    }
    return -1;
}
int main()
{
    vector<int> v{4,5,6,7,0,1,2};
    int tar = 0;
    cout << "Result: " << rotated_sorted(v, tar) << endl;
    return 0;
}

```

Output: ./search_4

Result: 4

- 1D Array - Reversal & Rotation

THEORY

- Reversal = two pointer swap from both ends moving inward. $O(n)$ time, $O(1)$ space.
- Rotation = cyclic shift of elements. Left rotate by k moves first k elements to end.
- The reversal trick solves rotation in $O(n)$ time $O(1)$ space — no extra array needed.
- Right rotate by k = left rotate by $n-k$. Same function, different argument.
- Always do $k = k \% n$ first — rotating by n is same as no rotation.

THE REVERSAL TRICK — why it works

Left rotate by k 3 reversals, $O(n)$, $O(1)$

// {1,2,3,4,5,6}, left rotate $k=2 \rightarrow$ {3,4,5,6,1,2}

$k = k \% n$;

reverse(v, 0, k-1); // reverse first k: {2,1,3,4,5,6}

reverse(v, k, n-1); // reverse rest: {2,1,6,5,4,3}

reverse(v, 0, n-1); // reverse all: {3,4,5,6,1,2} ✓

Right rotate by k same trick, different split

// {1,2,3,4,5,6}, right rotate $k=2 \rightarrow$ {5,6,1,2,3,4}

$k = k \% n$;

reverse(v, 0, n-k-1); // reverse first $n-k$: {4,3,2,1,5,6}

reverse(v, n-k, n-1); // reverse last k : {4,3,2,1,6,5}

reverse(v, 0, n-1); // reverse all: {5,6,1,2,3,4} ✓

// shortcut: right rotate k = left rotate $(n-k)$

HELPER FUNCTION — reuse everywhere

```
void reverse(vector<int>& v, int s, int e){
```

```
    while(s < e){ swap(v[s], v[e]); s++; e--; }
```

```
}
```

```
void leftRotate(vector<int>& v, int k){
```

```
    int n = (int)v.size();
```

```
    k = k % n;
```

```
    if(k == 0) return;
```

```
    reverse(v, 0, k-1);
```

```
    reverse(v, k, n-1);
```

```
    reverse(v, 0, n-1);
```

```
}
```

```
void rightRotate(vector<int>& v, int k){
```

```
    int n = (int)v.size();
```

```
    k = k % n;
```

```
    if(k == 0) return;
```

```
    reverse(v, 0, n-k-1);
```

```
    reverse(v, n-k, n-1);
```

```
    reverse(v, 0, n-1);
```

```
}
```

KEY INTERVIEW POINTS

Always do first $k = k \% n$ — handles $k \geq n$ and $k = 0$

Left rotate split first $k = [0, k-1]$, rest $= [k, n-1]$

Right rotate split first $n-k = [0, n-k-1]$, last $k = [n-k, n-1]$

Shortcut right rotate k = left rotate $n-k$

Reversal direction always swap from ends inward — never outward

Brute force extra array $O(n)$ space — mention but use reversal trick

COMMON MISTAKES

- Forgetting $k = k \% n$ — crashes or wrong answer for $k \geq n$
- Wrong split points — left rotate uses $k-1$ and k , right uses $n-k-1$ and $n-k$
- Hardcoding indices instead of using k — breaks for any k other than the test value
- Not handling $k=0$ after mod — wastes 3 reversals on no-op

COMPLEXITY

method	time	space
Brute force (extra array)	$O(n)$	$O(n)$
Reversal trick	$O(n)$	$O(1)$
One by one rotation	$O(n \times k)$	$O(1)$

PATTERN RECOGNITION

"Rotate array left by k "	→ reversal trick, 3 steps
"Rotate array right by k "	→ reversal trick or left rotate $n-k$
"Check if one array is rotation of another"	→ concatenate and search
"Find rotation count in rotated sorted array"	→ binary search for pivot
"Reverse words in a sentence"	→ reverse each word + reverse all

Q1. [Easy] Right rotate {1,2,3,4,5} by $k=2$.

Expected: {4,5,1,2,3}

```
#include <iostream>
#include <vector>
using namespace std;
void reverse(vector<int>& v, int s, int e)
{
    while(s < e)
    {
        swap(v[s], v[e]);
        s++;
        e--;
    }
}
void right_rotate(vector<int>& v, int k)
{
    int n = v.size();
    if(n == 0) return;
    k = k % n;
    if(k == 0) return;
    reverse(v, 0, n-k-1);
    reverse(v, n-k, n-1);
    reverse(v, 0, n-1);
}
int main()
{
    vector<int> v{1,2,3,4,5};
    int k = 2;
    right_rotate(v, k);
    cout << "Result:- ";
    for(int i=0; i<v.size(); i++)
        cout << v[i] << " ";
    cout << endl;
```

```

    return 0;
}
Output: ./rotate_1
Result:- 4 5 1 2 3

```

Q2. [Medium] Check if {3,4,5,1,2} is a rotation of {1,2,3,4,5}.

Expected: true

```

#include <iostream>
#include <vector>
using namespace std;
bool check_Rotated(vector<int>& v1, vector<int>& v2)
{
    int n = v1.size();
    int m = v2.size();
    if(n != m) return false;
    for(int i=0;i<m;i++)
        v2.push_back(v2[i]);
    for(int i=0;i<m;i++)
    {
        int j = 0;
        while(j < m && v2[i+j] == v1[j])
            j++;
        if(j == m) return true;
    }
    return false;
}
int main()
{
    vector<int> rv{3,4,5,1,2};
    vector<int> av{1,2,3,4,5};
    bool res = check_Rotated(rv, av);
    if(res)
        cout << "Rotated (true).\n";
    else
        cout << "Not Rotated (false).\n";
    return 0;
}
Output: ./rotate_2
Rotated (true).

```

Q3. [Medium] Given {1,2,3,4,5,6}, rotate left by k=9.

Expected: {4,5,6,1,2,3} ($k \% n = 9 \% 6 = 3$)

```

#include <iostream>
#include <vector>
using namespace std;
void reverse(vector<int>& v, int s, int e)
{
    while(s < e)
    {
        swap(v[s], v[e]);
        s++;
        e--;
    }
}
void rotate_left_9(vector<int>& v, int k)
{
    int n = v.size();
    if(n == 0) return;

```

```

    k = k%n;
    if(k == 0) return;
    reverse(v, 0, k-1);
    reverse(v, k, n-1);
    reverse(v, 0, n-1);
}
int main()
{
    vector<int> v{1,2,3,4,5,6};
    int k = 9;
    rotate_left_9(v, k);
    cout << "Result: ";
    for(int i=0;i<v.size();i++)
        cout << v[i] << " ";
    cout << endl;
    return 0;
}

```

Output: ./rotate_3

Result: 4 5 6 1 2 3

Q4. [Hard] Find how many times {4,5,6,7,0,1,2} was rotated.

Expected: 4 (pivot is at index 4)

```

#include <iostream>
#include <vector>
using namespace std;
int posIdx(vector<int>& v)
{
    int l = 0, r = v.size()-1;
    while(l <= r)
    {
        if(v[l] <= v[r])
            return l;
        int mid = (l + r) / 2;
        int next = (mid + 1)%v.size();
        int prev = (mid - 1 + v.size())%v.size();
        if(v[mid] <= v[next] && v[mid] <= v[prev])
            return mid;
        else if(v[mid] >= v[l])
            l = mid+1;
        else
            r = mid-1;
    }
    return 0;
}
int main()
{
    vector<int> v{4,5,6,7,0,1,2};
    cout << "Result of Index: " << posIdx(v) << endl;
    return 0;
}

```

Output: ./rotate_4

Result of Index: 4

-- 2D Arrays - Matrix traversal

Theory

A 2D array is stored **row by row in memory** (row-major order). `mat[i][j]` lives at base + $(i \times C + j) \times \text{size}$. The outer loop controls which dimension you visit first — swap it to change direction entirely.

5 Traversal Patterns

1. Row-wise — outer `i`, inner `j`. Visits left → right, row by row.

```
for(int i=0; i<R; i++)
    for(int j=0; j<C; j++)
        cout << mat[i][j];
```

2. Column-wise — outer `j`, inner `i`. Visits top → bottom, column by column.

```
for(int j=0; j<C; j++)
    for(int i=0; i<R; i++)
        cout << mat[i][j];
```

3. Diagonal — main diagonal where $i=j$, anti-diagonal where $i+j=C-1$.

```
for(int i=0; i<R; i++)
    cout << mat[i][i];    // main
for(int i=0; i<R; i++)
    cout << mat[i][C-1-i]; // anti
```

4. Spiral — shrink 4 boundaries inward. The two if guards are critical.

```
int top=0, bot=R-1, left=0, right=C-1;
while(top<=bot && left<=right){
    for(int c=left; c<=right; c++) print(top,c); top++;
    for(int r=top; r<=bot; r++) print(r,right); right--;
    if(top<=bot) for(int c=right; c>=left; c--) print(bot,c); bot--;
    if(left<=right) for(int r=bot; r>=top; r--) print(r,left); left++;
}
```

5. Transpose + Rotate — `j=i+1` avoids double-swapping.

```
// transpose
for(int i=0; i<R; i++)
    for(int j=i+1; j<C; j++)
        swap(mat[i][j], mat[j][i]);
// rotate 90° CW = transpose + reverse each row
// rotate 90° CCW = reverse each row + transpose
// rotate 180° = reverse each row + reverse all rows
```

1. [Easy] Print all elements of a 4×4 matrix in column-wise order.

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<vector<int>>> v{{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}, {13, 14, 15, 16}};
    int r = v.size();
    int c = v[0].size();
    for(int j=0; j<c; j++)
    {
        for(int i=0; i<r; i++)
            cout << v[i][j] << " ";
    }
}
```

```

        cout << endl;
        return 0;
    }
    Output: ./2d_traversal_1
    1 5 9 13 2 6 10 14 3 7 11 15 4 8 12 16

```

2. [Easy] Find sum of main diagonal elements.

```

#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<vector<int>> v{{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    int sum = 0;
    int r = v.size();
    int c = v[0].size();
    /*for(int i=0;i<r;i++) //2 Pass
    {
        for(int j=0;j<c;j++)
        {
            if(i == j)
                sum += v[i][j];
        }
    }*/
    for(int i=0;i<r;i++) // 1 Pass
    {
        sum += v[i][i];
    }
    cout << "Sum of main Diagonal:- " << sum << endl;
    return 0;
}
    Output: ./2d_traversal_2
    Sum of main Diagonal:- 15

```

3. [Medium] Print a 4×4 matrix in spiral order.

```

#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<vector<int>> v{{1,2,3,4}, {5,6,7,8}, {9,10,11,12}, {13,14,15,16}};
    int r = v.size();
    int c = v[0].size();
    int top = 0, bot = r-1, left = 0, right = c-1;
    while(top <= bot && left <= right)
    {
        for(int i=left;i<=right;i++)
            cout << v[top][i] << " ";
        top++;
        for(int i=top;i<=bot;i++)
            cout << v[i][right] << " ";
        right--;
        if(top<=bot)
        {
            for(int i=right;i>=left;i--)
                cout << v[bot][i] << " ";
            bot--;
        }
    }
}

```

```

        if(left<=right)
        {
            for(int i=bot;i>=top;i--)
                cout << v[i][left] << " ";
            left++;
        }
    }
    cout << endl;
    return 0;
}
Output: ./2d_traversal_3
1 2 3 4 8 12 16 15 14 13 9 5 6 7 11 10

```

4. [Medium] Rotate a 3×3 matrix 90° clockwise in-place.

```

#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<vector<int>> v{{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    int r = v.size();
    int c = v[0].size();
    for(int i=0;i<r;i++)
    {
        for(int j=i+1;j<r;j++)
        {
            swap(v[i][j], v[j][i]);
        }
    }
    for(int i=0;i<r;i++)
    {
        int x = 0, y = r-1;
        while(x < y)
        {
            swap(v[i][x], v[i][y]);
            x++;
            y--;
        }
    }
    cout << "Matrix after 90° clockwise in-place: ";
    for(int i=0;i<r;i++)
    {
        for(int j=0;j<r;j++)
        {
            cout << v[i][j] << " ";
        }
    }
    cout << endl;
    return 0;
}
Output: ./2d_traversal_4
Matrix after 90° clockwise in-place: 7 4 1 8 5 2 9 6 3

```

5. [Hard] Search target in a matrix where each row and column is sorted. Return {row, col} or {-1,-1}. Input: mat={{1,4,7},{2,5,8},{3,6,9}}, target=5 Expected: {1,1}

```

#include <iostream>
#include <vector>
using namespace std;

```

```

pair<int, int> target_Search(vector<vector<int>>& v, int tar)
{
    if(v.empty() || v[0].empty()) return {-1, -1};
    int n = v.size();
    int m = v[0].size();
    int i = 0, j = m-1;
    while(i < n && j >= 0)
    {
        if(v[i][j] == tar)
        {
            return {i, j};
        }
        else if(v[i][j] > tar)
            j--;
        else
            i++;
    }
    return {-1, -1};
}

int main()
{
    vector<vector<int>> v{{1,4,7},{2,5,8},{3,6,9}};
    int tar = 9;
    pair<int, int> x = target_Search(v, tar);
    cout << "r:" << x.first << " , c:" << x.second << endl;
    return 0;
}

```

Output: ./2d_traversal_5

r:2 , c:2

-- 2D Arrays - Matrix operations

Theory

A matrix operation transforms one or more 2D arrays into a result. Every operation has three things an interviewer cares about: the **dimension rule** (what sizes are allowed), the **formula** (what happens at each cell), and the **space trade-off** (can you do it in-place or do you need a new matrix).

The most important thing to understand before coding any matrix problem: **always extract R and C separately, never assume square.**

Operation 1 — Addition & Subtraction

Rule: Both matrices must be the same size. Result is same size. No tricks — just element-wise.

When asked in interview: Usually wrapped inside a bigger problem. Standalone addition is a warm-up question to test if you know the dimension rule.

// returns result matrix, does not modify inputs

```
vector<vector<int>> add(vector<vector<int>>& A, vector<vector<int>>& B)
```

```
{
    int R = (int)A.size(), C = (int)A[0].size();
    // always guard dimensions in interview
    if(R != (int)B.size() || C != (int)B[0].size()) return {};
    vector<vector<int>> res(R, vector<int>(C));
    for(int i = 0; i < R; i++)
        for(int j = 0; j < C; j++)
            res[i][j] = A[i][j] + B[i][j];
    return res;
}
```

// Time: $O(R \times C)$ Space: $O(R \times C)$

Key interview points:

- Always check dimensions before computing
- Return empty vector {} if dimensions mismatch — never crash
- Subtraction is identical, just change + to -

Operation 2 — Matrix Multiplication

The most important matrix operation. Appears in FAANG rounds, system design, and ML interviews.

Rule: A is $M \times N$, B is $N \times P \rightarrow C$ is $M \times P$. Inner dimensions must match (columns of A == rows of B).

Formula: $C[i][j] = \sum(A[i][k] * B[k][j])$ for $k = 0$ to $N-1$

```
vector<vector<int>> multiply(vector<vector<int>>& A, vector<vector<int>>& B)
```

```
{
    int M = (int)A.size();
    int N = (int)A[0].size(); // cols of A = rows of B
    int P = (int)B[0].size(); // cols of B
    if(N != (int)B.size()) return {}; // dimension guard
    vector<vector<int>> C(M, vector<int>(P, 0));
    for(int i = 0; i < M; i++)
        for(int j = 0; j < P; j++)
            for(int k = 0; k < N; k++)
                C[i][j] += A[i][k] * B[k][j];
    return C;
}
```

// Time: $O(M \times N \times P)$ Space: $O(M \times P)$

The loop variable rule — never mix these up:

- i = row of C and row of A
- j = col of C and col of B
- k = the shared dimension (col of A, row of B)

Operation 3 — Set Matrix Zeroes Google, Meta, Amazon

Rule: If $\text{mat}[i][j] == 0$, set entire row i and entire column j to zero. Must be done correctly — naive approach gives wrong answer.

Why naive fails: If you zero cells as you scan, a newly-zeroed cell looks like an original zero and incorrectly zeroes more rows/cols.

Correct approach — two passes:

```
void setZeroes(vector<vector<int>>& mat) {
    int R = (int)mat.size(), C = (int)mat[0].size();
    vector<bool> zeroRow(R, false), zeroCol(C, false);
    // pass 1: record which rows and cols have a zero
    for(int i = 0; i < R; i++)
        for(int j = 0; j < C; j++)
            if(mat[i][j] == 0) { zeroRow[i] = true; zeroCol[j] = true; }
    // pass 2: apply
    for(int i = 0; i < R; i++)
        for(int j = 0; j < C; j++)
            if(zeroRow[i] || zeroCol[j]) mat[i][j] = 0;
}
// Time:  $O(R \times C)$    Space:  $O(R+C)$ 
```

FAANG follow-up they always ask: "Can you do it in $O(1)$ space?" Answer: Use the first row and first column as marker arrays — store zero flags there instead of extra vectors. This is a common follow-up at Google and Meta.

Operation 4 — Maximal Square of 1s Google, Meta, Microsoft

Rule: Find the largest square containing only 1s. Return its area.

DP formula: $\text{dp}[i][j] = \min(\text{dp}[i-1][j], \text{dp}[i][j-1], \text{dp}[i-1][j-1]) + 1$ if $\text{mat}[i][j] == '1'$
 $\text{dp}[i][j]$ means the side length of the largest square whose bottom-right corner is at (i,j) .

```
int maximalSquare(vector<vector<char>>& mat)
{
    int R = (int)mat.size(), C = (int)mat[0].size();
    vector<vector<int>> dp(R, vector<int>(C, 0));
    int maxSide = 0;
    for(int i = 0; i < R; i++) {
        for(int j = 0; j < C; j++) {
            if(mat[i][j] == '1') {
                if(i == 0 || j == 0)
                    dp[i][j] = 1;
                else
                    dp[i][j] = min({dp[i-1][j], dp[i][j-1], dp[i-1][j-1]}) + 1;
                maxSide = max(maxSide, dp[i][j]);
            }
        }
    }
    return maxSide * maxSide; // area = side^2
}
```

// Time: $O(R \times C)$ Space: $O(R \times C)$

Why min of 3 neighbours? The square can only be as large as the smallest of the three adjacent squares allows. Think of it as the bottleneck — one short neighbour breaks the square.

Operation 5 — Number of Islands Amazon, Google, Microsoft

Rule: Count connected groups of '1's in a grid. Connection is 4-directional (up, down, left, right).

Approach: DFS flood fill — when you find a '1', DFS to mark all connected '1's as visited, then count that as one island.

```
void dfs(vector<vector<char>>& grid, int i, int j)
```

```

{
    int R = (int)grid.size(), C = (int)grid[0].size();
    if(i < 0 || i >= R || j < 0 || j >= C || grid[i][j] != '1') return;
    grid[i][j] = '0'; // mark visited by sinking the island
    dfs(grid, i+1, j);
    dfs(grid, i-1, j);
    dfs(grid, i, j+1);
    dfs(grid, i, j-1);
}

```

```
int numIslands(vector<vector<char>>& grid)
```

```

{
    int count = 0;
    for(int i = 0; i < (int)grid.size(); i++)
        for(int j = 0; j < (int)grid[0].size(); j++)
            if(grid[i][j] == '1') { dfs(grid, i, j); count++; }
    return count;
}

```

// Time: $O(R \times C)$ Space: $O(R \times C)$ call stack

Interview note: Mention that this modifies the input. If that's not allowed, use a separate visited[][] boolean matrix instead.

Operation 6 — Minimum Path Sum Google, Amazon

Rule: Find the path from top-left to bottom-right with minimum sum. Can only move right or down.

DP formula: $dp[i][j] = mat[i][j] + \min(dp[i-1][j], dp[i][j-1])$

```
int minPathSum(vector<vector<int>>& grid)
```

```

{
    int R = (int)grid.size(), C = (int)grid[0].size();
    // modify in-place to save space
    for(int i = 0; i < R; i++)
    {
        for(int j = 0; j < C; j++)
        {
            if(i == 0 && j == 0) continue;
            else if(i == 0) grid[i][j] += grid[i][j-1]; // only from left
            else if(j == 0) grid[i][j] += grid[i-1][j]; // only from top else
            grid[i][j] += min(grid[i-1][j], grid[i][j-1]);
        }
    }
    return grid[R-1][C-1];
}

```

// Time: $O(R \times C)$ Space: $O(1)$ in-place

1. [Easy — Flipkart, Amazon]

Add matrices $A = \{\{1,2,3\}, \{4,5,6\}, \{7,8,9\}\}$ and $B = \{\{9,8,7\}, \{6,5,4\}, \{3,2,1\}\}$

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
vector<vector<int>> add_2_mat(vector<vector<int>>& v1, vector<vector<int>>& v2)
```

```

{
    vector<vector<int>> res;
    int n = v1.size(), m = v1[0].size();
    if(n != v2.size() || m != v2[0].size()) return res;
    for(int i=0; i<n; i++)
    {
        vector<int> row;
        for(int j=0; j<m; j++)
        {
            row.push_back(v1[i][j] + v2[i][j]);
        }
    }
}

```

```

    }
    res.push_back(row);
}
return res;
}
int main()
{
    vector<vector<int>> A={{1,2,3},{4,5,6},{7,8,9}};
    vector<vector<int>> B={{9,8,7},{6,5,4},{3,2,1}};
    vector<vector<int>> res = add_2_mat(A, B);
    for(int i=0;i<res.size();i++)
    {
        for(int j=0;j<res[0].size();j++)
            cout << res[i][j] << " ";
        cout << endl;
    }
    return 0;
}

```

Output: ./2d_mat_ops_1

```

10 10 10
10 10 10
10 10 10

```

2. [Medium — Amazon, Microsoft]

Multiply $A = \{\{1,2\},\{3,4\}\}$ and $B = \{\{5,6\},\{7,8\}\}$ Expected: $\{\{19,22\},\{43,50\}\}$

```

#include <iostream>
#include <vector>
using namespace std;
vector<vector<int>> mat_product(vector<vector<int>>& v1, vector<vector<int>>& v2){
    vector<vector<int>> out;
    int n = v1.size(), m = v1[0].size();
    int x = v2.size(), y = v2[0].size();
    if(m != x) return out;
    out.resize(n, vector<int>(y, 0));
    for(int i=0;i<n;i++)
    {
        for(int j=0;j<y;j++)
            for(int k=0;k<m;k++)
                out[i][j] += v1[i][k]*v2[k][j];
    }
    return out;
}
int main()
{
    vector<vector<int>> A={{1,2},{3,4}};
    vector<vector<int>> B={{5,6},{7,8}};
    vector<vector<int>> res = mat_product(A, B);
    for(int i=0;i<res.size();i++)
    {
        for(int j=0;j<res[0].size();j++)
            cout << res[i][j] << " ";
        cout << endl;
    }
    return 0;
}

```

Output: ./2d_mat_ops_2

```

19 22
43 50

```

-- Array Techniques - Two pointer technique

Theory

A brute force approach checks every pair of indices with two nested loops. Two pointer eliminates the inner loop by using the **sorted order or structure of the data** to decide which pointer to move.

There are three flavours:

Flavour 1 — Opposite ends ($l=0$, $r=n-1$, move inward) Use when: working with sorted arrays, palindrome, pair sum, container problems.

Flavour 2 — Same direction ($slow=0$, $fast=0$, $fast$ runs ahead) Use when: remove duplicates, move zeros, partition, detect cycle.

Flavour 3 — Two arrays ($i=0$, $j=0$, both move forward independently) Use when: merge sorted arrays, compare sequences.

The Decision Rule

- Is the array sorted? → Opposite ends two pointer probably works
- Do I need to partition/filter in-place? → Same direction (slow/fast)
- Am I comparing two sequences? → Two array pointers

Flavour 1 — Opposite Ends

Pattern 1A — Pair with Target Sum

// array must be sorted

```
pair<int,int> pairSum(vector<int>& v, int target) {
    int l=0, r=(int)v.size()-1;
    while(l < r) {
        int sum = v[l] + v[r];
        if(sum == target) return {l, r};
        else if(sum < target) l++; // need bigger sum → move left forward
        else r--; // need smaller sum → move right back
    }
    return {-1, -1};
}
```

// Time: $O(n)$ Space: $O(1)$

// Brute force was $O(n^2)$ — two pointer eliminates inner loop

Pattern 1B — Container With Most Water

```
int maxWater(vector<int>& height) {
    int l=0, r=(int)height.size()-1, maxVol=0;
    while(l < r) {
        int vol = min(height[l], height[r]) * (r-l);
        maxVol = max(maxVol, vol);
        if(height[l] < height[r]) l++; // move the shorter side
        else r--;
    }
    return maxVol;
}
```

// Time: $O(n)$ Space: $O(1)$

Pattern 1C — Three Sum

Fix one element, run two pointer on the rest.

```
vector<vector<int>> threeSum(vector<int>& nums) {
    sort(nums.begin(), nums.end());
    vector<vector<int>> res;
    int n = (int)nums.size();
    for(int i=0; i<n-2; i++) {
        if(i>0 && nums[i]==nums[i-1]) continue; // skip duplicates
        int l=i+1, r=n-1;
        while(l < r) {
```

```

    int sum = nums[i]+nums[l]+nums[r];
    if(sum==0) {
        res.push_back({nums[i],nums[l],nums[r]});
        while(l<r && nums[l]==nums[l+1]) l++; // skip duplicates
        while(l<r && nums[r]==nums[r-1]) r--; // skip duplicates
        l++; r--;
    }
    else if(sum < 0) l++;
    else r--;
}
}
return res;
}
// Time: O(n2) Space: O(1) excluding output

```

Pattern 1D — Palindrome Check

```

bool isPalindrome(string& s) {
    int l=0, r=(int)s.length()-1;
    while(l < r) {
        if(s[l]!=s[r]) return false;
        l++; r--;
    }
    return true;
}
// Time: O(n) Space: O(1)

```

Pattern 1E — Trapping Rain Water

```

int trap(vector<int>& height) {
    int l=0, r=(int)height.size()-1;
    int maxL=0, maxR=0, water=0;
    while(l < r) {
        if(height[l] <= height[r]) {
            if(height[l] >= maxL) maxL=height[l];
            else water += maxL - height[l];
            l++;
        } else {
            if(height[r] >= maxR) maxR=height[r];
            else water += maxR - height[r];
            r--;
        }
    }
    return water;
}
// Time: O(n) Space: O(1) ← upgrade from your O(n) solution

```

Flavour 2 — Same Direction (Slow/Fast)

Pattern 2A — Remove Duplicates from Sorted Array

```

int removeDuplicates(vector<int>& v) {
    if(v.empty()) return 0;
    int j=0; // slow = write pointer
    for(int i=1; i<(int)v.size(); i++) { // fast = read pointer
        if(v[i] != v[j]) {
            j++;
            v[j] = v[i];
        }
    }
    return j+1;
}

```

// Time: O(n) Space: O(1)

Pattern 2B — Move Zeros

```
void moveZeros(vector<int>& v) {  
    int j=0;  
    for(int i=0; i<(int)v.size(); i++)  
        if(v[i]!=0) { swap(v[i],v[j]); j++; }  
}
```

// Time: O(n) Space: O(1)

Pattern 2C — Remove Element by Value

```
int removeElement(vector<int>& v, int val) {  
    int j=0;  
    for(int i=0; i<(int)v.size(); i++)  
        if(v[i]!=val) v[j++]=v[i];  
    return j;  
}
```

// Time: O(n) Space: O(1)

Pattern 2D — Linked List Cycle Detection

Fast pointer moves 2 steps, slow moves 1. If cycle exists, they meet.

```
bool hasCycle(ListNode* head) {  
    ListNode* slow=head, *fast=head;  
    while(fast && fast->next) {  
        slow=slow->next;  
        fast=fast->next->next;  
        if(slow==fast) return true;  
    }  
    return false;  
}
```

// Time: O(n) Space: O(1)

Pattern 2E — Find Middle of Array / Linked List

// middle of array — not really two pointer but same idea

```
int mid = l + (r-l)/2;
```

// middle of linked list — fast/slow

```
ListNode* middleNode(ListNode* head) {  
    ListNode* slow=head, *fast=head;  
    while(fast && fast->next) {  
        slow=slow->next;  
        fast=fast->next->next;  
    }  
    return slow; // slow is at middle when fast reaches end  
}
```

// Time: O(n) Space: O(1)

Flavour 3 — Two Arrays

Pattern 3A — Merge Two Sorted Arrays

```
void merge(vector<int>& v1, int m, vector<int>& v2, int n) {  
    int i=m-1, j=n-1, k=m+n-1;  
    while(i>=0 && j>=0)  
        v1[k--] = (v1[i]>v2[j]) ? v1[i--] : v2[j--];  
    while(j>=0) v1[k--]=v2[j--];  
}
```

// Time: O(m+n) Space: O(1)

Pattern 3B — Intersection of Two Arrays

```

vector<int> intersection(vector<int>& a, vector<int>& b) {
    sort(a.begin(), a.end());
    sort(b.begin(), b.end());
    vector<int> res;
    int i=0, j=0;
    while(i<(int)a.size() && j<(int)b.size()) {
        if(a[i]==b[j]) {
            if(res.empty() || res.back()!=a[i])
                res.push_back(a[i]);
            i++; j++;
        }
        else if(a[i]<b[j])
            i++;
        else
            j++;
    }
    return res;
}
// Time: O(n log n + m log m) Space: O(1) excluding output

```

1. [Easy] Given sorted array {1,2,3,4,6}, find pair that sums to 6.
 Expected: (2,4) at indices (1,3)

```

#include <iostream>
#include <vector>
using namespace std;
vector<pair<int, int>> two_sum(vector<int>& v, int x)
{
    vector<pair<int, int>> res;
    int n = v.size();
    if(n == 0) return res;
    int i = 0, j = n-1;
    while(i < j)
    {
        int sum = v[i]+v[j];
        if(sum == x)
        {
            res.push_back({v[i], v[j]});
            i++; j--;
        }
        else if (sum < x)
            i++;
        else
            j--;
    }
    return res;
}
int main()
{
    vector<int> v{0,1,2,3,4,6};
    int tar = 6;
    vector<pair<int, int>> res = two_sum(v, tar);
    if(!res.empty())
    {
        for(const auto& pair : res)
            cout << pair.first << " " << pair.second << endl;
    }
    return 0;
}

```

Output: ./2ptr_1

0 6

2 4

2. [Easy] Given "A man a plan a canal Panama", check if it is a palindrome ignoring spaces and case.

Expected: true

```
#include <iostream>
#include <string>
using namespace std;
char lower(char& c)
{
    if(c >= 'A' && c <= 'Z')
        c = c + 32;
    return c;
}
bool palindrome_str(string& str)
{
    //transform(str.begin(), str.end(), str.begin(), ::tolower);
    //#include <algorithm> // For std::transform
    //#include <cctype> // For std::tolower
    int i = 0, j = str.length()-1;
    while(i < j)
    {
        if(str[i] != ' ')
        {
            i++;
            continue;
        }
        if(str[j] != ' ')
        {
            j--;
            continue;
        }
        if(lower(str[i]) != lower(str[j]))
            return false;
        i++;
        j--;
    }
    return true;
}
int main()
{
    string s = "A man a plan a canal Panama";
    bool res = palindrome_str(s);
    if(res)
        cout << "String is palindrome: [" << s << "]" << endl;
    return 0;
}
```

Output: ./2ptr_2

String is palindrome: [A man a plan a canal Panama]

3. [Medium] Given {-1,0,1,2,-1,-4}, find all unique triplets summing to 0.

Expected: {-1,-1,2}, {-1,0,1}

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
```



```

vector<vector<int>> tripletSum(vector<int>& v, int t)
{
    vector<vector<int>> res;
    sort(v.begin(), v.end());
    for(int i=0;i<v.size()-2;i++)
    {
        if(i>0 && v[i] == v[i-1]) continue;
        int l = i+1, r = v.size()-1;
        while(l < r)
        {
            int sum = v[i]+v[l]+v[r];
            if(sum == t)
            {
                res.push_back({v[i], v[l], v[r]});
                while(l < r && v[l]==v[l+1])
                    l++;
                while(l < r && v[r]==v[r-1])
                    r--;
                l++;
                r--;
            }
            else if(sum < t)
                l++;
            else
                r--;
        }
    }
    return res;
}

int main()
{
    vector<int> v{-1,0,1,2,-1,-4};
    int tar = 0;
    vector<vector<int>> out = tripletSum(v, tar);
    for(int i=0;i<out.size();i++)
    {
        for(int j=0;j<out[0].size();j++)
            cout << out[i][j] << " ";
        cout << endl;
    }
    return 0;
}

```

Output: ./2ptr_3

-1 -1 2

-1 0 1

4. [Medium] Given height={1,8,6,2,5,4,8,3,7}, find max water container.

Expected: 49

```

#include <iostream>
#include <vector>
using namespace std;
int maxWater(vector<int>& v)
{
    int n = v.size();
    int l = 0, r = n-1;
    int maxVol = 0;
    while(l < r)
    {

```

```

        int vol = min(v[l], v[r])*(r-l);
        maxVol = max(maxVol, vol);
        if(v[l] < v[r])
            l++;
        else
            r--;
    }
    return maxVol;
}
int main()
{
    vector<int> v{1,8,6,2,5,4,8,3,7};
    cout << "Result (max water): " << maxWater(v) << endl;
    return 0;
}
Output: ./2ptr_4
Result (max water): 49

```

5. [Hard] Given two sorted arrays {1,2,4,5,6} and {2,3,4,5}, find their intersection.

Expected: {2,4,5}

```

#include<iostream>
#include<vector>
using namespace std;
vector<int> intersection(vector<int>v1,vector<int>v2)
{
    vector<int>res;
    int i=0;
    int j=0;
    while(i<v1.size() && j<v2.size())
    {
        if(v1[i]<v2[j])
            i++;
        else if(v1[i]>v2[j])
            j++;
        else
        {
            res.push_back(v1[i]);
            i++; j++;
        }
    }
    return res;
}
int main()
{
    vector<int> a{1,2,4,5,6};
    vector<int> b{2,3,4,5};
    vector<int> res = intersection(a, b);
    if(!res.empty())
    {
        for(int i=0;i<res.size();i++)
            cout << res[i] << " ";
        cout << endl;
    }
    return 0;
}
Output: ./2ptr_5
2 4 5

```

-- Array Techniques - Sliding Window Technique

Sliding window is a technique where you maintain a **contiguous subarray or substring** and slide it across the data — adding one element from the right, removing one from the left. It turns $O(n^2)$ or $O(n^3)$ brute force into $O(n)$.

Theory — The Core Idea

Brute force checks every possible subarray: two nested loops, $O(n^2)$. Sliding window avoids recomputing from scratch each time — it **reuses the previous window's computation** by adjusting for the one element added and one removed.

Fixed window — window size k never changes. Slide one step at a time. Use when: "subarray of size k ", " k consecutive elements"

Variable window — window grows and shrinks based on a condition. Use when: "longest/shortest subarray that satisfies X "

Fixed Window — Template

// find maximum sum subarray of size k

```
int maxSumFixed(vector<int>& v, int k) {
    int n = (int)v.size();
    if(n < k) return -1;
    // build first window
    int windowSum = 0;
    for(int i=0; i<k; i++) windowSum += v[i];
    int maxSum = windowSum;
    // slide: add right element, remove left element
    for(int i=k; i<n; i++) {
        windowSum += v[i];    // add incoming element
        windowSum -= v[i-k];  // remove outgoing element
        maxSum = max(maxSum, windowSum);
    }
    return maxSum;
}
// Time:  $O(n)$  Space:  $O(1)$ 
// Brute force was  $O(n \times k)$ 
```

The key operation: $\text{windowSum} += v[i] - v[i-k]$ — one line replaces the inner loop.

Variable Window — Template

// longest subarray with $\text{sum} \leq \text{target}$

```
int longestSubarray(vector<int>& v, int target) {
    int l=0, windowSum=0, maxLen=0;
    for(int r=0; r<(int)v.size(); r++) {
        windowSum += v[r];    // expand right
        while(windowSum > target)    // shrink left until valid
            windowSum -= v[l++];
        maxLen = max(maxLen, r-l+1);    // update answer
    }
    return maxLen;
}
```

// Time: $O(n)$ Space: $O(1)$

The template always has:

1. Expand right — add $v[r]$ to window
2. Shrink left — remove $v[l]$ and $l++$ while condition violated
3. Update answer — record result at valid window state

Pattern 1 — Max Sum Subarray of Size K

```
int maxSumK(vector<int>& v, int k) {
    int n=(int)v.size();
    int sum=0, maxSum=INT_MIN;
    for(int i=0; i<n; i++) {
        sum += v[i];
```

```

    if(i >= k-1) {
        maxSum = max(maxSum, sum);
        sum -= v[i-k+1]; // remove element going out of window
    }
}
return maxSum;
}
// Input: {2,1,5,1,3,2}, k=3 → Output: 9 (subarray {5,1,3})
// Time: O(n) Space: O(1)

```

Pattern 2 — Longest Substring Without Repeating Characters

```

int lengthOfLongest(string s) {
    unordered_set<char> window;
    int l=0, maxLen=0;
    for(int r=0; r<(int)s.length(); r++) {
        while(window.count(s[r])) // duplicate found — shrink
            window.erase(s[l++]);
        window.insert(s[r]);
        maxLen = max(maxLen, r-l+1);
    }
    return maxLen;
}
// Input: "abcabcbb" → Output: 3 ("abc")
// Time: O(n) Space: O(min(n, charset))

```

Pattern 3 — Minimum Window Substring

```

string minWindow(string s, string t) {
    unordered_map<char,int> need, have;
    for(char c : t) need[c]++;
    int formed=0, required=(int)need.size();
    int l=0, minLen=INT_MAX, start=0;

    for(int r=0; r<(int)s.length(); r++) {
        char c = s[r];
        have[c]++;
        // check if this char's frequency satisfies requirement
        if(need.count(c) && have[c]==need[c]) formed++;

        // shrink window while all requirements met
        while(formed == required) {
            if(r-l+1 < minLen) { minLen=r-l+1; start=l; }
            have[s[l]]--;
            if(need.count(s[l]) && have[s[l]] < need[s[l]]) formed--;
            l++;
        }
    }
    return minLen==INT_MAX ? "" : s.substr(start, minLen);
}
// Input: s="ADOBECODEBANC", t="ABC" → Output: "BANC"
// Time: O(|s|+|t|) Space: O(|t|)

```

Pattern 4 — Maximum of Every Window of Size K

```

vector<int> maxSlidingWindow(vector<int>& v, int k) {
    deque<int> dq; // stores indices, front = index of current max
    vector<int> res;
    for(int i=0; i<(int)v.size(); i++) {
        // remove indices outside current window
        while(!dq.empty() && dq.front() <= i-k)

```

```

    dq.pop_front();
    // remove smaller elements — they can never be max
    while(!dq.empty() && v[dq.back()] <= v[i])
        dq.pop_back();
    dq.push_back(i);
    // window has k elements starting from index k-1
    if(i >= k-1) res.push_back(v[dq.front()]);
}
return res;
}

```

// Input: {1,3,-1,-3,5,3,6}, k=3 → Output: {3,3,5,5,6}

// Time: O(n) Space: O(k)

Why deque? It maintains a decreasing sequence of useful candidates. Front is always the max of the current window. Elements smaller than the new element can never be the max — so they are discarded.

Pattern 5 — Longest Substring with At Most K Distinct Characters

```

int longestKDistinct(string s, int k) {
    unordered_map<char,int> freq;
    int l=0, maxLen=0;
    for(int r=0; r<(int)s.length(); r++) {
        freq[s[r]]++;
        while((int)freq.size() > k) { // too many distinct chars
            freq[s[l]]--;
            if(freq[s[l]]==0) freq.erase(s[l]);
            l++;
        }
        maxLen = max(maxLen, r-l+1);
    }
    return maxLen;
}

```

// Input: "araaci", k=2 → Output: 4 ("araa")

// Time: O(n) Space: O(k)

Pattern 6 — Subarray Product Less Than K

```

int numSubarrayProductLessThanK(vector<int>& v, int k) {
    if(k <= 1) return 0;
    int l=0, product=1, count=0;
    for(int r=0; r<(int)v.size(); r++) {
        product *= v[r];
        while(product >= k) product /= v[l++];
        count += r-l+1; // all subarrays ending at r with left in [l,r]
    }
    return count;
}

```

// Input: {10,5,2,6}, k=100 → Output: 8

// Time: O(n) Space: O(1)

Why r-l+1? At each valid window [l,r], the number of valid subarrays ending at r is exactly r-l+1 (all starting points from l to r).

Pattern 7 — Permutation in String

```

bool checkInclusion(string p, string s) {
    if(p.length() > s.length()) return false;
    int freq[26]={0};
    for(char c : p) freq[c-'a']++;
    int k=(int)p.length(), matches=0;
    // count how many chars already match
    for(int i=0; i<26; i++) if(freq[i]==0) matches++;
}

```

```

for(int r=0; r<(int)s.length(); r++) {
    // add right char
    int c=s[r]-'a';
    freq[c]--;
    if(freq[c]==0) matches++;
    // remove left char when window exceeds k
    if(r >= k) {
        int lc=s[r-k]-'a';
        if(freq[lc]==0) matches--;
        freq[lc]++;
        if(freq[lc]==0) matches++;
    }
    if(matches==26) return true;
}
return false;
}
// Input: p="ab", s="eidbaooo" → Output: true ("ba" at index 3)
// Time: O(|s|) Space: O(1)

```

1. [Easy — Amazon, Flipkart]

Max sum subarray of size k=3.

Input: {2,1,5,1,3,2}

Expected: 9

```

#include <iostream>
#include <climits>
#include <vector>
using namespace std;
int maxi(vector<int>& v, int k)
{
    int maxi = INT_MIN, sum = 0;
    int n = v.size();
    for(int i=0;i<n;i++)
    {
        sum += v[i];
        if(i>=k-1)
        {
            maxi = max(maxi, sum);
            sum -= v[i-k+1];
        }
    }
    return maxi;
}
int main()
{
    vector<int> v{2,1,5,1,3,2};
    int k = 3;
    cout << "Max Sum Subarray: " << maxi(v, k) << endl;
    return 0;
}

```

Output: ./sl_i_win_1

Max Sum Subarray: 9

2. [Easy — Amazon, Flipkart]

Smallest subarray with sum >= 7.

Input: {2,3,1,2,4,3}

Expected: 2 (subarray {4,3})

```

#include <iostream>

```

```

#include <vector>
#include <climits>
using namespace std;
int small_subarray(vector<int>& v, int k)
{
    int n = v.size();
    int sum = 0, min_len = INT_MAX;
    int l = 0;
    for(int i=0;i<n;i++)
    {
        sum += v[i];
        while(sum >= k)
        {
            min_len = min(min_len, i-l+1);
            sum -= v[l];
            l++;
        }
    }
    return (min_len == INT_MAX) ? 0: min_len;
}
int main()
{
    vector<int> v{2,3,1,2,4,3};
    int sum = 7;
    cout << "Result:- " << small_subarray(v, sum) << endl;
    return 0;
}

```

Output: ./sli_win_2

Result:- 2

3. [Medium — Google, Meta, Amazon]

Longest substring without repeating chars.

Input: "pwwkew"

Expected: 3 ("wke")

4. [Medium — Google, Amazon]

Longest substring with at most 2 distinct chars.

Input: "araaci"

Expected: 4 ("araa")

5. [Medium — Amazon, Google]

Max of every window of size k=3.

Input: {1,3,-1,-3,5,3,6}

Expected: {3,3,5,5,6}

6. [Hard — Google, Meta, Amazon]

Minimum window substring.

Input: s="ADOBECODEBANC", t="ABC"

Expected: "BANC"

7. [Hard — Google, Meta]

Permutation in string.

Input: p="ab", s="eidbaooo"

Expected: true

6. Linked Lists

-- Singly Linked List

Step 1: Foundations – Node, List, Basic Traversal

Before you manipulate lists, understand how a linked list works:

1. **Node structure** -> **done**
 - Create a Node struct/class with data and next.
2. **Create a simple list**
 - Manually link 2–3 nodes.
3. **Traversal**
 - Write a function to traverse and print the list.
4. **Edge cases**
 - Empty list, single node list.

Goal: You should be able to create and print a linked list.

Step 2: Basic Insertions

Focus on adding nodes:

1. **insertAtStart** -> **done**
2. **insertAtEnd** -> **done**
3. **insertAtPosition** -> **done**

Tip: Think about:

- Updating head
- Traversing to position (for middle insert)
- Handling empty list

Goal: Can insert nodes anywhere in the list safely.

Step 3: Basic Deletions

Now remove nodes safely:

1. **deleteFromStart** -> **done**
2. **deleteFromEnd** -> **done**
3. **deleteAtPosition** -> **done**
4. **deleteByValue** -> **done**
5. **deleteAllOccurrences** -> **done**

Tip: Always handle **head node** and **empty list** cases. Free memory properly if using C++.

Goal: Can delete any node without breaking the list.

Step 4: Simple Queries & Utilities

Learn to **analyze and read the list**:

1. **countNodes** – number of nodes -> **done**
2. **search** – find a value (iterative & recursive) -> **done**
3. **traverse** – print or process nodes -> **done**
4. **reverse** – reverse the list (iterative first) -> **done**
5. **freeList** – safely delete entire list -> **done**

Goal: You can inspect, count, and reverse lists confidently.

Step 5: Extra Handy Operations

Add more advanced yet common operations:

1. **insertAfterValue** – insert after a node with a given value -> **done**
2. **removeDuplicates** – works for sorted or unsorted lists -> **done**
3. **reverseInPairs** – swap nodes pairwise -> **done**
4. **findMiddle** – fast/slow pointer trick -> **done**
5. **detectLoop** – Floyd's cycle detection -> **done**
6. **Start of the Cycle** - Floyd's cycle detection -> **done**
7. **Find Cycle Length** - Floyd's cycle detection -> **done**
8. **mergeSortedLists** – merge two sorted lists into one -> **done**

Tip: Use **dummy nodes** for easier insertion/deletion edge cases.

Goal: Handle practical manipulations and interview-style tasks.

Step 6: Advanced Manipulations

Learn harder problems that involve **sublist operations, rotations, and intersections**:

1. **Rotate list (by k positions)** – left or right rotation
2. **Reverse sublist (between m and n)** -> **done**
3. **Split list (into two halves)**
4. **Check palindrome** – using stack or fast/slow pointer
5. **Remove N-th node from end** – two-pointer method
6. **Find intersection of two lists**
7. **Sort linked list** – merge sort or insertion sort

Tip: Most of these require careful pointer management and sometimes **helper pointers/dummy nodes**.

Goal: Comfortable with pointer tricks, two-pointer technique, and advanced operations.

Step 7: Queries & Properties

Analyze the list and gather information:

1. **Find max/min node** -> **done**
2. **Find nth node from start or end** -> **done**
3. **Check for loop and remove it** -> **done**
4. **Length of the list** -> **done**
5. **Print in reverse without modifying** -> **done**
6. **Count occurrences** -> **done**
7. **Sum of all nodes** -> **done**

Tip: Combine traversal with counters or recursion for elegant solutions.

Goal: Extract all useful information from the list efficiently.

Step 8: Variants / Patterns

Learn **common linked list patterns**, often asked in interviews:

1. **Odd-even reordering**
2. **Pairwise swap nodes**
3. **Flatten list of lists**
4. **Skip M nodes and delete N nodes**
5. **Segregate 0s, 1s, 2s**

Tip: These are mostly about pointer manipulation **without using extra memory**.

Goal: Master tricky pointer-based patterns.

Step 9: Common Interview Techniques

Focus on **general techniques used in almost every linked list question**:

1. **Fast/slow pointer** – middle node, cycle detection
2. **Dummy node usage** – simplifies edge cases
3. **Recursive vs iterative implementations**
4. **Using stack or extra memory** – reversing, palindrome
5. **Intersection point**
6. **Add two numbers**
7. **Clone with random pointer**

Tip: Learn to **visualize pointers** on paper. Most interview questions are about **pointer manipulation** rather than loops.

Goal: You can tackle **any linked list interview question** confidently.

**** Single Linked List Function Code**

//Create Node

```
Node* createNode(int val)
{
    Node* newNode = new Node();
    newNode->data = val;
    newNode->next = NULL;
    return newNode;
}
```

//Insert at Start

```
void insertAtStart(Node*& head, int val)
{
    Node* newNode = new Node();
    newNode->data = val;
    newNode->next = head;
    head = newNode;
}
```

//Insert at End

```
void insertAtEnd(Node*& head, int val)
{
    Node* newNode = new Node();
    newNode->data = val;
    newNode->next = NULL;
    if(!head)
    {
        head = newNode;
        return;
    }
    Node* temp = head;
    while(temp->next != NULL)
    {
        temp = temp->next;
    }
    temp->next = newNode;
}
```

//Insert at given position

```
void insertAtPosition(Node*& head, int val, int pos)
{
    if(pos == 1)
    {
        insertAtStart(head, val);
        return;
    }
    Node* newNode = new Node();
    newNode->data = val;
    Node* temp = head;
    for(int i=1; i<pos-1 && temp; i++)
    {
        temp = temp->next;
    }
    if(!temp)
    {
        cout << "Position out of Range.\n";
        delete newNode;
        return;
    }
}
```

```

    }
    newNode->next = temp->next;
    temp->next = newNode;
}

```

//Insert value after target value

```

void insertAfterValue(Node*& head, int tar, int val)
{
    if(!head)
    {
        cout << "Linked List is Empty.\n";
        return;
    }
    Node* newNode = new Node();
    newNode->data = val;
    Node* temp = head;
    while(temp != NULL)
    {
        if(temp->data == tar)
        {
            newNode->next = temp->next;
            temp->next = newNode;
            return;
        }
        temp = temp->next;
    }
    cout << "Target Value not found.\n";
}

```

//Delete From beginning

```

void deleteFromBeg(Node*& head)
{
    if(!head)
        return;
    Node* temp = head;
    head = head->next;
    delete temp;
}

```

//Delete from end

```

void deleteFromEnd(Node*& head)
{
    if(!head)
        return;
    if(!head->next)
    {
        delete head;
        head = NULL;
        return;
    }
    Node* temp = head;
    while(temp->next->next)
    {
        temp = temp->next;
    }
    delete temp->next;
    temp->next = NULL;
}

```

//Delete from position

```
void deleteFromPos(Node*& head, int pos)
{
    if(!head)
        return;
    if(pos == 1)
    {
        deleteFromBeg(head);
        return;
    }
    Node* temp = head;
    for(int i=1;i<pos-1 && temp; i++)
    {
        temp = temp->next;
    }
    if(!temp || !temp->next)
    {
        cout << "Position out of range.\n";
        return;
    }
    Node* delNode = temp->next;
    temp->next = delNode->next;
    delete delNode;
}
```

//Delete By Value

```
void deleteByValue(Node*& head, int val)
{
    if(!head)
    {
        cout << "Linked List is Empty.\n";
        return;
    }
    if(head->data == val)
    {
        Node* temp = head;
        head = head->next;
        delete temp;
        return;
    }

    Node* curr = head;
    while(curr->next != NULL && curr->next->data != val)
    {
        curr = curr->next;
    }
    if(curr->next == NULL)
    {
        cout << "Value not found in list.\n";
        return;
    }
    Node* temp = curr->next;
    curr->next = curr->next->next;
    delete temp;
    return;
}
```

//Delete All Occurance(Value)

```
void deleteAllOccurance(Node*& head, int val)
{
    if(!head)
    {
        cout << "Linked List is Empty.\n";
        return;
    }
    if(head != NULL && head->data == val)
    {
        Node* temp = head;
        head = head->next;
        delete temp;
    }
    Node* curr = head;
    while(curr != NULL && curr->next != NULL)
    {
        if(curr->next->data == val)
        {
            Node* temp = curr->next;
            curr->next = curr->next->next;
            delete temp;
        }
        else
        {
            curr = curr->next;
        }
    }
}
```

//Remove Duplicates (Sorted or Unsorted)

```
void removeDub(Node*& head)
{
    unordered_set<int> seen;
    Node* curr = head;
    Node* prev = NULL;
    while(curr)
    {
        if(seen.find(curr->data) != seen.end())
        {
            prev->next = curr->next;
            delete curr;
            curr = prev->next;
        }
        else
        {
            seen.insert(curr->data);
            prev = curr;
            curr = curr->next;
        }
    }
}
```

//Count Node

```
int countNode(Node*& head)
{
    if(!head)
    {
```

```

        cout << "Linked List is Empty.\n";
        return 0;
    }
    int cnt = 0;
    Node* temp = head;
    while(temp != NULL)
    {
        cnt++;
        temp = temp->next;
    }
    return cnt;
}

```

//Search element iterator

```

int idxSearch(Node*& head, int val)
{
    if(head == NULL)
    {
        cout << "Linked List is Empty.\n";
        return -1;
    }
    int idx = 1;
    Node* temp = head;
    while(temp != NULL)
    {
        if(temp->data == val)
        {
            return idx;
        }
        temp = temp->next;
        idx++;
    }
    return -1;
}

```

//Search element recursive

Pending ...

//Process & Print Linked list

```

void showLL(Node* head)
{
    if(head == NULL)
    {
        cout << "Linked List is empty.\n";
        return;
    }
    Node* temp = head;
    while(temp != NULL)
    {
        cout << temp->data << "->";
        temp = temp->next;
    }
    cout << "NULL\n";
}

```

// Reverse Linked List

```

void reverseList(Node*& head)
{

```

```

Node* prev = NULL;
Node* curr = head;
Node* next = NULL;
while(curr)
{
    next = curr->next;
    curr->next = prev;
    prev = curr;
    curr = next;
}
head = prev;
}

```

// Free List

```

void freeList(Node*& head)
{
    while(head)
    {
        Node* temp = head;
        head = head->next;
        delete temp;
    }
}

```

//Find Max Node

```

int maxNode(Node*& head)
{
    if(!head)
        return INT_MIN;
    int maxVal = head->data;
    Node* temp = head->next;
    while(temp)
    {
        if(temp->data > maxVal)
            maxVal = temp->data;
        temp = temp->next;
    }
    return maxVal;
}

```

//Find Min Node

```

int minNode(Node*& head)
{
    if(!head)
        return INT_MAX;
    int minVal = head->data;
    Node* temp = head->next;
    while(temp)
    {
        if(temp->data < minVal)
            minVal = temp->data;
        temp = temp->next;
    }
    return minVal;
}

```

//Sum of all Nodes

```

int sumNode(Node*& head)

```

```

{
    if(!head)
    {
        cout << "Linked List is Empty.\n";
        return -1;
    }
    int sum = 0;
    Node* temp = head;
    while(temp)
    {
        sum += temp->data;
        temp = temp->next;
    }
    return sum;
}

```

//Print in reverse without modifying

//1. Recursive Approach

```

void reverse_recursive(Node* head)
{
    if(!head)
    {
        cout << "Linked List is Empty.\n";
        return;
    }
    reverse_recursive(head->next);
    cout << head->data << "->";
}

```

//2. Iterative Approach using a Stack

```

void reverse_stack(Node* head)
{
    if(!head)
    {
        cout << "Linked List is Empty.\n";
        return;
    }
    stack<int> s;
    Node* temp = head;
    while(temp)
    {
        s.push(temp->data);
        temp = temp->next;
    }
    while(!s.empty())
    {
        cout << s.top();
        s.pop();
        if(!s.empty())
            cout << "->";
        else
            cout << "->NULL\n";
    }
}

```

// Occurance Count

```

void cntOccure(Node* head)
{

```



```

    if(!head)
    {
        cout << "Linked List is Empty.\n";
        return;
    }
    unordered_map<int, int> freq;
    Node* temp = head;
    while(temp)
    {
        freq[temp->data]++;
        temp = temp->next;
    }
    for(auto& f : freq)
        cout << f.first << " " << f.second << endl;
}

```

// Find nth node from start or end

```

pair<int, int> nthNode(Node* head, int pos)
{
    if(!head)
    {
        cout << "Linked List is Empty.\n";
        return {-1, -1};
    }
    int len = countNode(head);
    if(len < pos)
    {
        cout << "Position is Wrong.\n";
        return {-1, -1};
    }
    int start = INT_MIN;
    int end = INT_MIN;
    int l = 1;
    Node* temp = head;
    while(temp != NULL && l < pos)
    {
        l++;
        temp = temp->next;
    }
    start = temp->data;
    int j = 0;
    Node* temp1 = head;
    while(j < (len-pos))
    {
        j++;
        temp1 = temp1->next;
    }
    end = temp1->data;
    return {start, end};
}

```

// Find middle element

//Using Length of linked list

```

int midEle(Node* head)
{
    if(!head || !head->next)
    {
        cout << "Linked List is Empty or Single node.\n";
    }
}

```

```

        return -1;
    }
    int l = countNode(head);
    if(l%2 == 0)
        l = l-1;
    Node* temp = head;
    int i = 0;
    for(i=0;i<l/2;i++)
    {
        temp = temp->next;
    }
    return temp->data;
}

```

//Using slow/fast of linked list

```

int middleEle(Node* head)
{
    if(!head || !head->next)
    {
        cout << "Linked List is Empty or Single node.\n";
        return -1;
    }
    Node* slow = head;
    Node* fast = head->next;
    while(fast && fast->next)
    {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow->data;
}

```

//Cycle / loop detection

```

bool hasCycle(Node* head)
{
    if(!head)
    {
        cout << "Linked List is Empty.\n";
        return false;
    }
    Node* slow = head;
    Node* fast = head;
    while(fast && fast->next)
    {
        slow = slow->next;
        fast = fast->next->next;
        if(slow == fast)
        {
            return true;
        }
    }
    return false;
}

```

// Find start of loop

```

int loopStart(Node* head)
{
    if(!head)

```

```

{
    cout << "Linked List is Empty.\n";
    return INT_MIN;
}
Node* slow = head;
Node* fast = head;
while(fast && fast->next)
{
    slow = slow->next;
    fast = fast->next->next;
    if(slow == fast)
    {
        slow = head;
        while(slow != fast)
        {
            slow = slow->next;
            fast = fast->next;
        }
        return slow->data;
    }
}
return INT_MIN;
}

```

// Find length of loop

```

int loopLen(Node* head)
{
    if(!head)
    {
        cout << "Linked List is Empty.\n";
        return 0;
    }
    Node* slow = head;
    Node* fast = head;
    while(fast && fast->next)
    {
        slow = slow->next;
        fast = fast->next->next;
        if(slow == fast)
        {
            int len = 1;
            Node* temp = slow->next;
            while(temp != slow)
            {
                len++;
                temp = temp->next;
            }
            return len;
        }
    }
    return 0;
}

```

// Detect loop and delete

```

void delLoop(Node*& head)
{
    if(!head)
    {

```

```

        cout << "LInked List is Empty.\n";
        return;
    }
    Node* slow = head;
    Node* fast = head;
    bool isloop = false;
    while(fast && fast->next)
    {
        slow = slow->next;
        fast = fast->next->next;
        if(slow == fast)
        {
            cout << "Loop Detected in linked List.\n";
            isloop = true;
            break;
        }
    }
    if(!isloop)
        return;
    slow = head;
    if(slow == fast)
    {
        while(fast->next != slow)
            fast = fast->next;
        fast->next = NULL;
        return;
    }
    while(slow->next != fast->next)
    {
        slow = slow->next;
        fast = fast->next;
    }
    fast->next = NULL;
    return;
}

```

// Merge 2 Sorted Lists

```

Node* mergeSort(Node* head, Node* head1)
{
    if(head == NULL)
        return head1;
    if(head1 == NULL)
        return head;
    if(head->data <= head1->data)
    {
        head->next = mergeSort(head->next, head1);
        return head;
    }
    else
    {
        head1->next = mergeSort(head, head1->next);
        return head1;
    }
}

```

// Reverse in Pair

```

void reverseInPair(Node*& head)
{

```

```

if(!head || !head->next)
{
    cout << "Linked List is Empty or Single node.\n";
    return;
}
Node* curr = head;
Node* prev = NULL;
head = head->next;
while(curr && curr->next)
{
    Node* nextPair = curr->next->next;
    Node* second = curr->next;

    second->next = curr;
    curr->next = nextPair;

    if(prev)
        prev->next = second;
    prev = curr;
    curr = nextPair;
}
}

```

// Reverse sublist (between m and n)

```

void reverse_sublist(Node*& head, int m, int n)
{
    int len = countNode(head);
    if(!head || n > len || n < m)
    {
        cout << "Linked List is Empty or last value is more then list length.\n";
        return;
    }
    Node* prev = NULL;
    Node* curr = head;
    Node* next = NULL;
    int pos = 1;
    while(pos < m)
    {
        prev = curr;
        curr = curr->next;
        pos++;
    }
    Node* sub = prev;
    Node* tail = curr;
    for(int i=0;i<=n-m;i++)
    {
        next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    if(sub)
        sub->next = prev;
    else
        head = prev;
    tail->next = curr;
    return;
}

```

// Check palindrome – using stack

```
bool checkPalindrome(Node* head)
{
    if(!head)
        return false;
    stack<int> s;
    Node* temp = head;
    while(temp != NULL)
    {
        s.push(temp->data);
        temp = temp->next;
    }
    temp = head;
    while(temp != NULL)
    {
        if(temp->data != s.top())
            return false;
        else
            s.pop();
        temp = temp->next;
    }
    return true;
}
```

// Check palindrome – using fast/slow pointer

```
bool palindromeCheck(Node* head)
{
    if(!head) return false;
    Node* slow = head;
    Node* fast = head;
    while(fast && fast->next)
    {
        slow = slow->next;
        fast = fast->next->next;
    }
    Node* prev = NULL;
    Node* curr = slow;
    while(curr)
    {
        Node* next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    Node* first = head;
    Node* second = prev;
    while(second)
    {
        if(first->data != second->data)
            return false;
        first = first->next;
        second = second->next;
    }
    return true;
}
```